



INFORME DE PROYECTO INTEGRADOR

Asignatura: Estructura de Datos

SMARTCAMPUS UTA WEB

Desarrollo de un aplicativo web para gestión de atención, trámites, organización documental y rutas internas, aplicando estructuras de datos avanzadas y arquitectura de software

Carrera: Ingeniería de Software
Asignatura: Estructura de Datos
Tipo de trabajo: Proyecto integrador avanzado
Modalidad: Aplicativo web con base de datos y arquitectura por capas
Docente: Ing. José Caiza Caizabuano, Mg.

Estudiantes:

León Tusa Jair Alexander
Medina Chico Darlyn Monserrath
Miranda Bano Rolando Sebastián
Moyolema Criollo Katherine Lissette
Ortiz Cunalata Alina Elizabeth

Período Académico: Enero – Julio 2026



1. Introducción

En la formación profesional de la Carrera de Software, el estudio y dominio de las estructuras de datos adquiere su máximo valor cuando se aplica dentro de soluciones de software completas, robustas y funcionales, superando los enfoques tradicionales de ejercicios aislados de programación. El proyecto integrador «**SmartCampus UTA Web**» surge como una propuesta metodológica y técnica alineada con la planificación académica de la asignatura Estructura de Datos.

Este sistema propone el diseño, modelado y construcción de un aplicativo web integral capaz de centralizar y optimizar procesos críticos dentro del campus universitario: la gestión de turnos de atención al usuario, la tramitación administrativa, la organización jerárquica de documentos institucionales y el cálculo de rutas de navegación interna mediante grafos. La implementación real de estructuras de datos lineales y no lineales se acopla directamente con una arquitectura de software por capas y un almacenamiento moderno basado en servicios en la nube (BaaS), garantizando la integridad, mantenibilidad y escalabilidad del ecosistema digital del campus.

2. Justificación

El desarrollo de este proyecto se justifica plenamente desde tres ejes fundamentales: el técnico, el académico y el institucional.

- **Ámbito técnico:** Demuestra la viabilidad de acoplar estructuras de datos abstractas implementadas de forma nativa (Listas, Pilas, Colas, Árboles y Grafos) con tecnologías modernas de persistencia como Supabase (PostgreSQL) y arquitecturas desacopladas en JavaScript.
- **Perspectiva académica:** Consolida el aprendizaje práctico de algoritmos de ordenamiento, estructuras autorreferenciales, recursividad y recorridos óptimos, aplicados a casos de uso de la vida real.
- **Nivel institucional:** Atiende una problemática latente en los centros de educación superior: la fragmentación de servicios de atención y la desorientación física o administrativa de los estudiantes. Al unificar estos flujos en una sola interfaz interactiva, se mitigan los tiempos de espera y se incrementa radicalmente la transparencia de los procesos institucionales.

3. Problema a Resolver

En el entorno actual de la Universidad Técnica de Ambato, la atención a los estudiantes en ventanillas, el seguimiento individualizado de trámites académicos, la clasificación jerárquica de la documentación de las dependencias y la orientación geográfica para localizar laboratorios u oficinas se gestionan de manera dispersa, manual o mediante sistemas aislados que no comparten información.

Esta fragmentación operativa provoca:

- Cuellos de botella significativos en los períodos de matriculación y solicitudes de alta demanda.
- Pérdida de trazabilidad sobre el estado real de los documentos.
- Confusión en la comunidad universitaria al intentar trasladarse de un punto a otro dentro del campus físico.

La ausencia de una plataforma centralizada que relacione la información de los usuarios, sus solicitudes temporales, la jerarquía de las oficinas y la topología del campus representa una deficiencia tecnológica que el presente sistema busca resolver mediante un diseño de software unificado y eficiente.



4. Tema del Proyecto

Desarrollo del aplicativo web «**SmartCampus UTA Web**» orientado a la gestión centralizada de atención al usuario, seguimiento de trámites, organización jerárquica documental y cálculo de rutas óptimas internas del campus universitario, aplicando estructuras de datos dinámicas avanzadas, servicios en la nube Supabase y arquitectura limpia por capas en el cliente y servidor.

5. Objetivo General

Diseñar e implementar un aplicativo web integral denominado **SmartCampus UTA Web** que optimice los procesos de atención por turnos, flujos de trámites, jerarquías de dependencias y navegación interna mediante el uso explícito de estructuras de datos lineales y no lineales, garantizando la persistencia de datos relacional y una arquitectura modular escalable.

6. Objetivos Específicos

1. Levantar y documentar de manera formal los requerimientos funcionales y no funcionales que rigen el funcionamiento de la gestión universitaria dentro de un campus inteligente.
2. Modelar los procesos del negocio mediante diagramas de casos de uso y esquemas de flujo que definan con precisión la interacción de los diferentes actores con el sistema.
3. Diseñar y normalizar una base de datos relacional en la plataforma Supabase (PostgreSQL), aplicando restricciones de integridad referencial, claves primarias y foráneas distribuidas en módulos lógicos.
4. Codificar en JavaScript estructuras de datos personalizadas (Cola, Pila, Lista Simple, Lista Doble, Lista Circular, Árbol N-Ario y Grafo) sin depender de librerías externas de colecciones para resolver las necesidades operativas de la lógica del negocio.
5. Desarrollar una interfaz gráfica web intuitiva y *responsive* empleando HTML5, CSS3 y Bootstrap 5, estructurada bajo una arquitectura por capas bien definidas que separe la presentación, la lógica de servicios y el acceso a la base de datos.
6. Validar la estabilidad y el rendimiento del sistema mediante pruebas unitarias y de integración de las colas de turnos y algoritmos de rutas, asegurando la sincronización en tiempo real con Supabase.

7. Alcance del Proyecto

El sistema SmartCampus UTA comprende la implementación de un entorno web funcional que abarca cinco módulos integrados:

- Seguridad y Autenticación
- Gestión de Turnos en Ventanilla
- Seguimiento de Trámites Académicos
- Clasificación de Documentos por Dependencias
- Mapa Interactivo del Campus



Tecnológicamente, el alcance cubre el desarrollo completo del *frontend*, la capa lógica de estructuras de datos que se ejecutan en memoria dinámicamente para agilizar los procesos de ordenamiento y filtrado, y la vinculación asíncrona bidireccional con Supabase como motor de datos relacional remoto. No se contempla en esta fase la integración con pasarelas de pago externas ni sistemas de facturación.

8. Requerimientos Iniciales

8.1. Requerimientos Funcionales

Código	Descripción
RF1	Autenticación y Control de Acceso: El sistema debe permitir el registro e inicio de sesión seguro de usuarios con cifrado, validando los roles asignados (Estudiante, Empleado, Administrador, Docente, Personal de Secretaría) mediante tokens de sesión dinámicos.
RF2	Gestión de Turnos en Tiempo Real: Permitir a los estudiantes solicitar turnos virtuales para atención en ventanillas específicas, insertando la solicitud en una cola de atención.
RF3	Asignación Rotativa de Ventanillas (Round-Robin): El sistema debe distribuir automáticamente los turnos de la cola entre los empleados disponibles en las ventanillas activas de forma cíclica y equitativa.
RF4	Registro y Seguimiento de Trámites: Los estudiantes podrán iniciar solicitudes o trámites adjuntando metadatos básicos. Cada trámite debe registrar un historial detallado de estados (Iniciado, En Revisión, Aprobado, Rechazado).
RF5	Organización Jerárquica Documental: Permitir al administrador configurar la estructura arbórea de las dependencias universitarias (Facultades, Carreras, Departamentos) y clasificar los documentos institucionales dentro de dicha jerarquía.
RF6	Mapa de Navegación del Campus: El sistema debe mostrar de forma interactiva el mapa del campus, permitiendo seleccionar un punto de origen y uno de destino para calcular y trazar la ruta física más óptima entre edificios y laboratorios.
RF7	Bitácora de Auditoría (Logs del Sistema): Registrar de forma inmutable cada acción crítica realizada por los usuarios (creación de trámites, atención de turnos, modificaciones) para su posterior revisión jerárquica e histórica en orden cronológico inverso.

Cuadro 1: Requerimientos funcionales del sistema SmartCampus UTA Web.

8.2. Requerimientos No Funcionales

RNF1 — Arquitectura de Software por Capas: El código de la aplicación debe estar estructurado en capas independientes (Presentación, Servicios/Negocio, Estructuras de Datos y Persistencia) para asegurar la modularidad y el desacoplamiento técnico.



RNF2 — Sincronización e Integridad Cloud: Toda modificación en el estado de los datos debe sincronizarse de manera asíncrona con el proveedor BaaS Supabase, manteniendo la integridad referencial de la base de datos PostgreSQL.

RNF3 — Interfaz Adaptable (*Responsive Design*): Las interfaces de usuario de todos los *dashboards* deben desarrollarse con un diseño fluido que se adapte correctamente a dispositivos móviles, *tablets* y ordenadores de escritorio utilizando Bootstrap 5 y variables de estilos globales en CSS.

RNF4 — Gestión Eficiente de Memoria: Las estructuras de datos implementadas localmente deben optimizar los tiempos de respuesta del cliente web, evitando fugas de memoria y reduciendo las llamadas redundantes a la base de datos remota mediante la retención estructurada en memoria volátil de las jerarquías y mapas estáticos.

9. Diseño y Construcción de la Base de Datos

La base de datos relacional para el sistema SmartCampus UTA Web ha sido diseñada siguiendo un riguroso proceso de normalización hasta la Tercera Forma Normal (3FN), eliminando redundancias y dependencias transitivas anómalas. Se aloja directamente en Supabase, aprovechando la robustez y escalabilidad del motor relacional PostgreSQL. El modelo está estructurado en catorce entidades lógicas interconectadas por restricciones explícitas de integridad referencial, agrupadas en los siguientes sub-módulos operativos:

- **Módulo de Usuarios y Seguridad:** Compuesto por las tablas *Usuario*, *Rol* y la tabla intermedia *UsuarioRol* (relación muchos a muchos), permitiendo un control granular de accesos. Las entidades especializadas *Estudiante* y *Empleado* se desprenden mediante relaciones de herencia lógica con integridad uno a uno (1:1) referenciando la clave primaria del usuario base.
- **Módulo de Turnos y Atención:** Estructurado a través de *TipoServicio* y *Turno*, mapeando campos críticos de estado, hora de emisión, hora de atención y la ventanilla asignada para el control algorítmico del flujo.
- **Módulo de Trámites y Flujos de Trabajo:** Controlado por las entidades *TipoTramite*, *Tramite* y *SeguimientoTramite*. Esta última actúa como un histórico inmutable que guarda marcas de tiempo y descripciones detalladas por cada transición de estado físico de la solicitud.
- **Módulo de Organización Documental e Institucional:** Compuesto por las tablas *Dependencia* y *CategoriaDocumento*, las cuales implementan relaciones autorreferenciales (campos tipo *IdPadre*) para dar soporte directo a la estructura jerárquica del Árbol N-Ario, vinculando los registros de la tabla *Documento*.
- **Módulo de Auditoría:** Centrado en la tabla de solo lectura *HistorialAccion*, encargada de persistir secuencialmente los logs operacionales del software.

10. Arquitectura de Software Propuesta

Para garantizar una separación estricta de responsabilidades, alta mantenibilidad y un desarrollo paralelo eficiente entre los miembros del equipo, el sistema implementa una **arquitectura por capas desacoplada** (N-Tier Client-Side & BaaS Architecture):



Capa	Responsabilidad principal	Ejemplos / Implementación
Presentación (Frontend)	Interactuar con el usuario, renderizar componentes visuales, capturar eventos y realizar validaciones en el cliente.	Páginas HTML5 con Bootstrap 5 y hojas de estilos personalizadas (<code>main.css</code>).
Lógica de Negocio (Service Layer)	Encapsular las reglas del negocio universitario e invocar coordinadamente los algoritmos de estructuras de datos y las operaciones asíncronas de datos.	Módulos JavaScript controladores de los paneles de control (<code>dashboards/</code>).
Estructuras de Datos Avanzadas	Gestionar en memoria volátil los datos recuperados de la nube, permitiendo encolamiento, desapilado, ordenación y recorridos de árboles o grafos sin sobrecargar la red.	Centralizada en <code>utils/structures.js</code> .
Persistencia (Data Access Layer)	Canalizar solicitudes desde la lógica de negocio y mapear resultados JSON del servicio BaaS mediante llamadas asíncronas (<code>async/await</code>).	Cliente Supabase (<code>supabase-config.js</code>), <code>data.js</code> y <code>auth.js</code> .
Datos (Database Layer)	Almacenamiento definitivo, garantizando persistencia, evaluación de <i>constraints</i> e integridad referencial de los esquemas.	Motor relacional PostgreSQL en la infraestructura de Supabase.

Cuadro 2: Capas de la arquitectura de SmartCampus UTA Web.

11. Integración de Estructuras de Datos en el Sistema

El sistema define a nivel de código tres tipos de nodos como unidad fundamental dinámica:

- **Nodo Simple:** Contiene el dato y un puntero hacia el nodo siguiente. Es la base arquitectónica para las Pilas y Colas.
- **Nodo Doble:** Añade un puntero adicional hacia el nodo anterior, utilizado para permitir navegación bidireccional en las Listas Dobles.
- **Nodo Árbol:** Contiene el dato, un arreglo dinámico de referencias a los hijos y una referencia al nodo padre, utilizado para sostener jerarquías organizacionales.

11.1. Aplicación de las Estructuras de Datos

- **Cola Dinámica $\langle T \rangle$ (FIFO — Módulo de Turnos):** Garantiza equidad en la atención estudiantil. Cuando un estudiante solicita un turno, se ejecuta el método `enqueue()` agregándolo al final de la estructura. El personal de secretaría lo consume extrayéndolo desde el frente (`dequeue()`). Un árbol de decisión automatiza además si se asigna el turno directamente a la ventanilla con menor carga o si el usuario debe esperar.



- **Lista Simple $\langle T \rangle$ (Trámites y Certificados):** Colección dinámica sin memoria contigua que permite inserción y búsqueda eficiente, ideal para gestionar los registros de becas y certificados de los estudiantes sin un límite predefinido de tamaño.
- **Lista Doble $\langle T \rangle$ (Expedientes):** Gracias a su doble puntero (anterior y siguiente), permite al usuario avanzar o retroceder intuitivamente a través de las diferentes fases históricas en el visor de expedientes sin necesidad de recorrer repetitivamente toda la lista desde la cabeza.
- **Pila Dinámica $\langle T \rangle$ (LIFO — Historial y Bitácora):** Registra cada acción operativa del usuario apilando (**push**) el evento. Esto facilita la recuperación del evento más reciente al tope de la pila, útil para funciones como un botón «Deshacer».
- **Lista Circular Simplemente Enlazada $\langle T \rangle$:** Modela el algoritmo Round-Robin para balanceo de carga. Al no poseer un final, permite la rotación automática, equitativa y sin fin de las cargas de trabajo entre las cuatro ventanillas activas de atención (Secretaría, Finanzas, Académico y Bienestar Estudiantil).
- **Árbol N-Ario (Organización Documental):** Cada nodo puede tener múltiples hijos, modelando con exactitud la jerarquía institucional. El Árbol N-Ario posee una raíz central (Nivel 0: «Documentos UTA») que se subdivide en áreas como «Académico» (con hijos de Nivel 2: Reglamentos, Mallas Curriculares, Sílabus) y «Administrativo» (Formularios, Actas, Financiero). Se emplea un recorrido recursivo en Pre-orden (visitando la raíz y luego descendiendo por los hijos de izquierda a derecha) para renderizar el explorador de archivos con correcta indentación visual.
- **Grafo Dirigido Ponderado (Mapa Campus Huachi):** El campus se representa como un grafo no dirigido y ponderado. Consta de **21 vértices** (edificios y puntos clave) y **30 aristas** (conexiones peatonales con peso en metros). Se emplea una Lista de Adyacencia para almacenar las relaciones, lo cual ahorra significativamente memoria al tratarse de una topología dispersa.

11.2. Algoritmos de Búsqueda y Recorridos en Grafos

Sobre la topología del Campus Huachi, se evaluaron dos enfoques algorítmicos:

- **Búsqueda en Anchura (BFS):** Optimiza el número de saltos — complejidad $O(V + E)$. Este recorrido por niveles, que utiliza una Cola subyacente, es ideal para rutas simples donde los tramos se consideran de igual valor.
- **Algoritmo de Dijkstra:** Optimiza la distancia métrica real — complejidad $O(V^2)$. Es el algoritmo implementado para la navegación de los estudiantes, ya que considera los pesos de las aristas en metros. Como caso de prueba, el algoritmo calcula la ruta óptima desde FISEI hacia el ESTADIO: **FISEI** → **Civil** → **Contabilidad** → **Universidad** → **Estadio**, con una distancia total de **380 metros**.



12. Implementación del Aplicativo Web

12.1. Tecnologías Empleadas

Capa	Tecnología	Justificación
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5	Permite una interfaz responsiva y rápida. Bootstrap acelera el desarrollo con componentes estándar.
Backend / BaaS	Supabase	Proporciona autenticación, base de datos PostgreSQL en la nube y sincronización en tiempo real mediante WebSockets.
Estructuras de Datos	JavaScript vanilla (structures.js)	Implementación nativa de todas las estructuras sin librerías externas, maximizando la comprensión algorítmica.
Mapa Interactivo	Leaflet.js + OpenStreetMap	Visualización geoespacial del Campus Huachi con coordenadas GPS reales y renderizado de rutas óptimas.
Base de datos	PostgreSQL (vía Supabase)	Sistema relacional robusto, gratuito y compatible. Soporta consultas complejas requeridas por el módulo de grafos.
Control de versiones	Git + GitHub	Permite trabajo colaborativo, trazabilidad de cambios y respaldo en la nube.

12.2. Estructura del Proyecto

El proyecto sigue una arquitectura modular organizada en directorios claramente diferenciados:

- /utils/structures.js — Implementación nativa de todas las estructuras de datos.
- /utils/supabase-config.js — Configuración y cliente del servicio BaaS.
- /services/data.js, auth.js — Abstracción del acceso a datos.
- /dashboards/ — Controladores de los paneles de control por módulo.
- /pages/ — Vistas HTML de cada módulo funcional.
- /assets/css/main.css — Estilos globales y variables de diseño.

13. Metodología de Trabajo

El proyecto se desarrolló siguiendo una metodología por fases, adaptada al contexto del curso. A continuación se detallan las actividades y productos esperados por fase:



Fase	Actividades	Producto esperado
Inicio y análisis	Definición del problema, alcance, actores, requerimientos funcionales y no funcionales, casos de uso.	Documento de análisis aprobado.
Diseño	Modelo de datos relacional, arquitectura por capas, diagrama de clases UML, prototipo de interfaz.	Diseño técnico y script de base de datos.
Construcción	Implementación de estructuras de datos personalizadas, backend (controladores, servicios, repositorios), frontend (vistas, layouts, JS), e integración con BD.	Aplicativo web operativo con 5 módulos funcionales.
Pruebas	Validación funcional (casos de prueba), pruebas técnicas (rendimiento, estrés), revisión de consistencia de datos.	Casos de prueba documentados y evidencias de ejecución.
Cierre	Elaboración de informe final, manual de usuario, manual técnico, preparación de defensa oral.	Informe final, manuales, repositorio finalizado y exposición.

14. Resultados Obtenidos

14.1. Resumen de Estructuras Implementadas

Estructura	Módulo	Operación clave	Complejidad	Integrante
Cola (FIFO)	Turnos	encolar / desencolar	$O(1)$	Alina
Pila (LIFO)	Historial	apilar / desapilar	$O(1)$	Alina
Lista Simple	Trámites	insertar / buscar	$O(n)$	Rolando
Lista Doble	Expedientes	avanzar / retroceder	$O(1)$	Rolando
Lista Circular	Ventanillas	rotar (Round-Robin)	$O(1)$	Rolando
Árbol N-Ario	Documentos	insertar / preorden	$O(n)$	Monserrath
Grafo (Lista Ady.)	Mapa campus	agregarNodo / Arista	$O(1)$	Katherine
BFS	Ruta (saltos)	búsqueda en anchura	$O(V + E)$	Jair
Dijkstra	Ruta (distancia)	camino mínimo	$O(V^2)$	Jair

Cuadro 3: Estructuras de datos implementadas en SmartCampus UTA Web.



14.2. Comparativa BFS vs. Dijkstra

Criterio	BFS	Dijkstra
Optimiza	Número de saltos	Distancia total
Complejidad	$O(V + E)$	$O(V^2)$
Uso de cola	Sí (FIFO)	Cola de prioridad
Requiere pesos	No	Sí
Aplicación	Ruta con menos paradas	Ruta más corta en metros

15. Capturas de Funcionamiento del Aplicativo

A continuación se presentan las capturas de pantalla que evidencian el correcto funcionamiento de cada módulo del sistema SmartCampus UTA Web.

15.1. Pantalla de Inicio / Registro de Usuario

La pantalla de creación de cuenta permite el registro mediante correo institucional @uta.edu.ec, validando el formato en tiempo real. El formulario solicita nombres, apellidos, usuario institucional, rol en la institución, contraseña con requisitos de seguridad (mínimo 8 caracteres, mayúsculas, minúsculas, números y símbolo) y confirmación de contraseña.

Figura 1: Formulario de creación de cuenta con validación de correo institucional.

15.2. Verificación de Correo Institucional

Tras el registro, el sistema envía un código de 6 dígitos al correo institucional del usuario. La pantalla de verificación implementa un temporizador de expiración del código y permite reenviar en caso de no recibirse, asegurando que solo usuarios con cuenta institucional válida accedan al sistema.

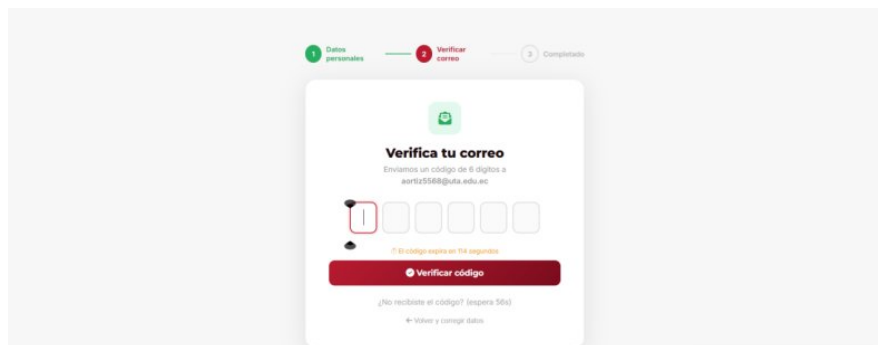


Figura 2: Pantalla de verificación de correo con código de 6 dígitos y temporizador.

15.3. Inicio de Sesión (Login)

La pantalla de inicio de sesión solicita el correo institucional y la contraseña. Incluye opción de recuperación de contraseña y enlace para registrar una nueva cuenta, siguiendo patrones de UX estándar para aplicaciones institucionales.

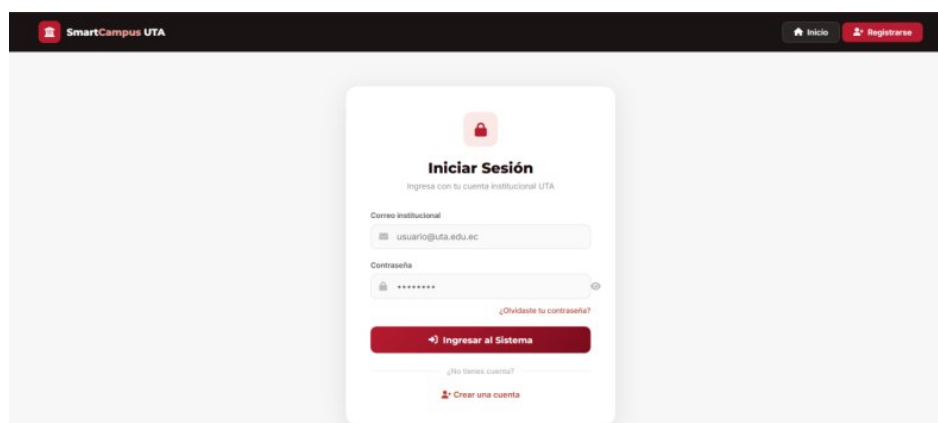


Figura 3: Pantalla de inicio de sesión con acceso mediante cuenta institucional UTA.

15.4. Dashboard Principal — Gestión de Trámites

El panel de control principal (*dashboard*) muestra un resumen personalizado para el usuario autenticado: número de trámites activos, turnos activos y solicitudes pendientes. La sección «Mis Últimos Trámites» lista los trámites recientes con código, tipo, fecha y estado. La sección «Mis Turnos Activos» muestra los turnos vigentes con opción de crear nuevos. Este módulo implementa internamente una **Lista Simple** para la gestión dinámica de trámites y una **Cola FIFO** para los turnos en espera.

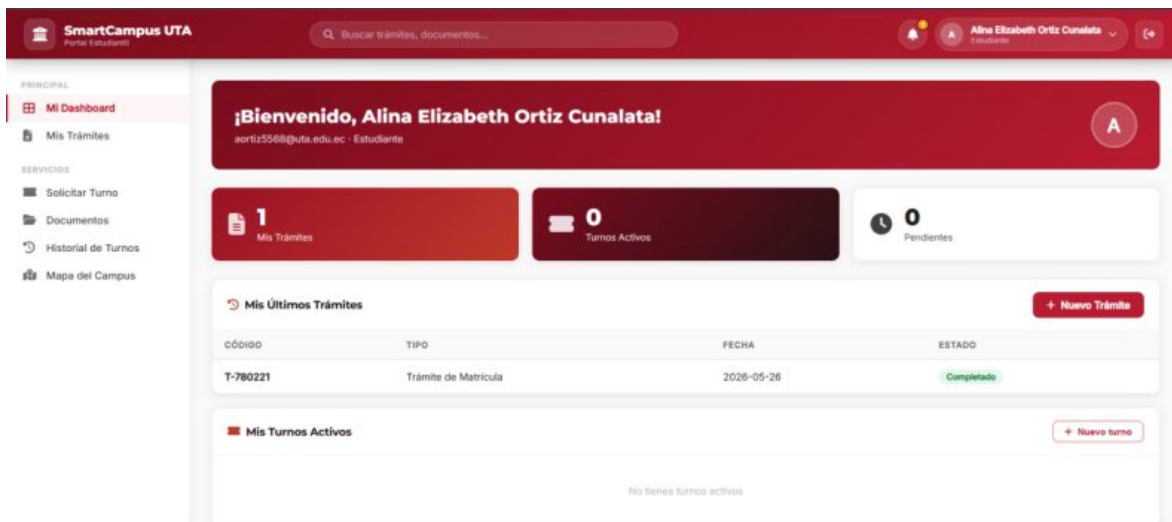


Figura 4: Dashboard principal mostrando resumen de trámites, turnos activos y pendientes del usuario.

15.5. Módulo de Gestión de Trámites (Lista Simple)

La vista «Mis Trámites» presenta el historial y seguimiento de todas las solicitudes del usuario. Cada trámite muestra su código único, tipo, fecha de creación, estado actual (Completado, En Revisión, Pendiente, Rechazado) y descripción asociada. La estructura de datos subyacente es una **Lista Simplemente Enlazada** que permite inserción dinámica y búsqueda eficiente por ID.



Figura 5: Módulo «Mis Trámites» con listado de solicitudes, estados y descripción (Lista Simple).

15.6. Módulo de Atención por Turnos (Cola FIFO)

El formulario de solicitud de turno permite seleccionar el tipo de servicio (Secretaría, Finanzas, Académico o Bienestar), la fecha de la cita (Lunes a Viernes, 08:00–13:00 y 14:00–17:00), la hora disponible y el motivo de la consulta. Al confirmar, el turno se agrega al final de la **Cola FIFO** correspondiente al servicio seleccionado. La asignación rotativa entre ventanillas se gestiona mediante una **Lista Circular** con algoritmo Round-Robin.



Figura 6: Formulario de solicitud de turno presencial con selección de servicio, fecha y hora (Cola FIFO).

15.7. Módulo de Clasificación Documental (Árbol N-Ario)

El módulo «Documentos» implementa un explorador de categorías basado en un **Árbol N-Ario** con recorrido en Pre-orden. El panel izquierdo muestra el árbol de categorías con jerarquía completa: *Documentos UTA* → Académico (Reglamentos, Mallas Curriculares, Sílabus) → Administrativo (Formularios, Actas) → Financiero. Al seleccionar una categoría, el panel derecho lista los documentos correspondientes. Cada nodo puede tener n hijos, modelando con exactitud la estructura organizacional de la universidad.

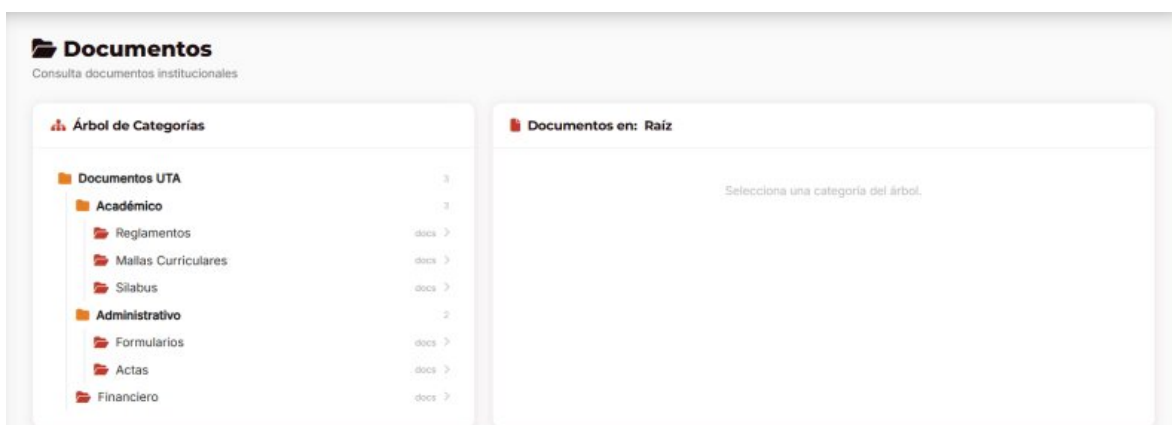


Figura 7: Explorador de documentos institucionales con árbol de categorías jerárquico (Árbol N-Ario).

15.8. Módulo de Mapa de Rutas Internas (Grafo Ponderado + Dijkstra)

El módulo «Mapa del Campus» visualiza el Campus Huachi de la Universidad Técnica de Ambato mediante Leaflet.js y OpenStreetMap. El mapa muestra los 21 nodos (edificios y puntos de interés) clasificados por categoría: Administración, Facultades y Servicios. El panel derecho «Calcular Ruta» permite seleccionar un origen y un destino; al confirmar, el algoritmo de **Dijkstra** calcula y traza la ruta peatonal óptima en metros sobre el mapa interactivo, considerando los pesos de las 30 aristas del **Grafo Ponderado**.

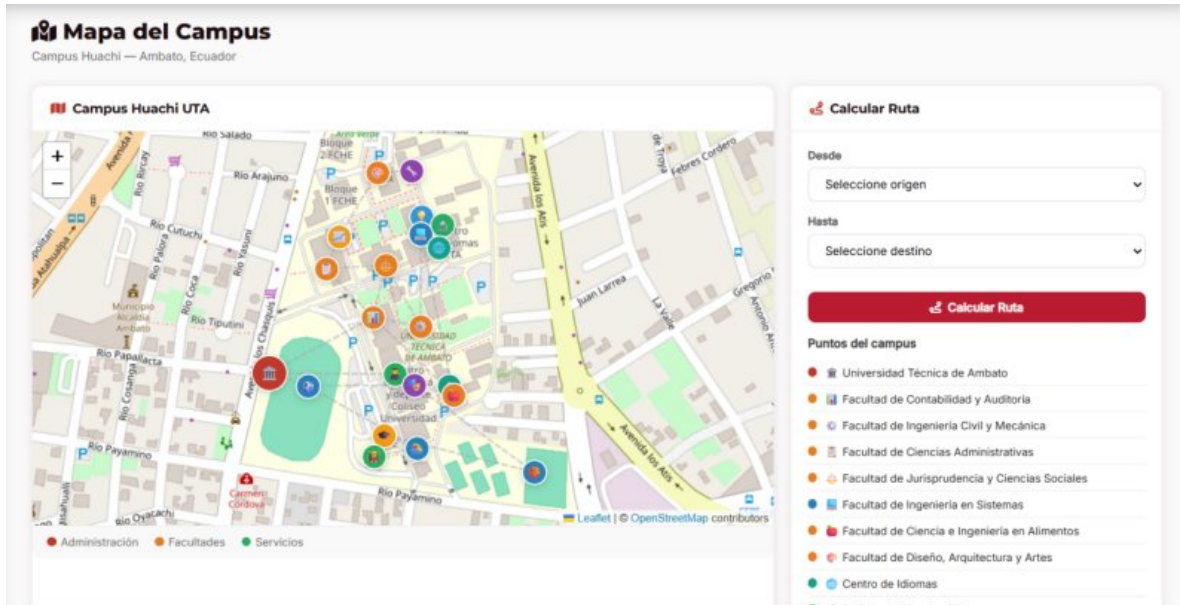


Figura 8: Mapa interactivo del Campus Huachi con calculadora de rutas óptimas (Grafo + Dijkstra).

16. Conclusiones

- El proyecto SmartCampus UTA demostró que las estructuras de datos son herramientas prácticas que resuelven problemas reales: la Cola garantiza equidad en la atención de turnos, la Pila permite deshacer acciones, las Listas Enlazadas ofrecen flexibilidad dinámica para trámites y expedientes, y el Grafo con Dijkstra optimiza la navegación física del campus.
- La arquitectura por capas desacoplada (Presentación, Servicios, Estructuras de Datos y Persistencia) garantizó la mantenibilidad del código y facilitó el trabajo colaborativo entre los cinco integrantes del grupo, reduciendo las dependencias entre módulos.
- La implementación desde cero de todas las estructuras en `structures.js`, sin librerías externas, fortaleció la comprensión profunda de los mecanismos internos de cada estructura, su complejidad temporal y sus casos de uso óptimos [1].
- La elección de la representación del grafo como lista de adyacencia (en lugar de matriz de adyacencia) tuvo un impacto directo en el rendimiento: al ser el Campus Huachi un grafo disperso (30 de las 210 aristas posibles), la lista de adyacencia resultó más eficiente en uso de memoria.
- La integración de Supabase como servicio BaaS permitió una sincronización bidireccional en tiempo real, reduciendo la complejidad del backend y permitiendo al equipo concentrarse en la implementación de las estructuras de datos y la lógica de negocio.

17. Recomendaciones

- Implementar una **cola de prioridad** (*heap*) para optimizar el algoritmo de Dijkstra de $O(V^2)$ a $O((V + E) \log V)$, relevante si el mapa del campus escala a cientos de nodos.



- Incorporar un **árbol AVL o árbol B** en el módulo de búsqueda de trámites para reducir la complejidad de $O(n)$ a $O(\log n)$ cuando el volumen de registros sea alto.
- Explorar la integración de **algoritmos A*** como alternativa a Dijkstra en el mapa del campus, aprovechando las coordenadas GPS como heurística para acelerar la búsqueda de rutas.
- Documentar el código con **JSDoc** y agregar pruebas unitarias para cada estructura en `structures.js`, garantizando la mantenibilidad del proyecto a largo plazo [2].
- Extender el sistema para soportar **notificaciones en tiempo real** vía Supabase Realtime cuando un turno sea atendido, eliminando la necesidad de que el estudiante refresque manualmente la página.

18. Referencias

- [1] R. Sedgewick and K. Wayne, *Algorithms*. Upper Saddle River, NJ: Addison-Wesley, 4th ed., 2011.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA: MIT Press, 3rd ed., 2009.

19. Anexos

Link: https://github.com/Monse1806/SMARTCAMPUS_UTA.git